```
const { importData, movies, server } = require('./provider')

...

'Has a movie with specific ID': (parameters) => {
   movies.getFirstMovie().id = parameters.id;
   return Promise.resolve({ description: `Movie with ID
${parameters.id} added!` });
 }
```

The `getFirstMovie` function returns the first movie, and we're simply updating its ID with the ID that the consumer has used as part of their contract, represented by `parameters.id`. For your reference, the file provider/provider-contract.spec.js file should now look like this:

```
const { Verifier } = require('@pact-foundation/pact');
const { importData, movies, server } = require('./provider')

const port = '3001';
const app = server.listen(port, () => console.log(`Listening
on port ${port}...`));

importData();

const options = {
  provider: 'MoviesAPI',
  providerBaseUrl: `http://localhost:${port}`,
  pactBrokerUrl: process.env.PACT_BROKER_BASE_URL,
  pactBrokerToken: process.env.PACT_BROKER_TOKEN,
  providerVersion: '1.0.0',     #1
  publishVerificationResult: true,
  consumerVersionTags: ['main'],
  stateHandlers: {
    'Has a movie with specific ID': (parameters) => {
      movies.getFirstMovie().id = parameters.id;
      return Promise.resolve({ description: `Movie with
ID ${parameters.id} added!` });
    }
  }
};

const verifier = new Verifier(options);

describe('Pact Verification', () => {
  test('should validate the expectations of movie-consumer', () => {
    return verifier
      .verifyProvider()
      .then(output => {
        console.log('Pact Verification Complete!');
        console.log('Result:', output);
        app.close();
      })
  });
});
```

#1 As the provider isn't uploading the contract, the version remains similar. In chapter 10, we'll use an automated version number.